Search Application Report

Magnus Moi Tytlandsvik
UiT Arctic University of Norway
Tromsø, Norway
matyt7723@uit.no

I. Introduction

In this project we will implement a search application supporting complex search queries over multiple documents using Boolean logic operators. To achieve this we have implemented an inverted index.

II. Theoretical Background

A. Inverted Index

An inverted index is a data structure that excels in retrieving documents that contain a term or a set of terms. [1] In contrast to a forward index, an inverted index stores a mapping from content to documents, making it optimal for search engines, database systems, etc. [2]

B. Recursive Descent Parser

A recursive descent parser is a top-down parser that parses input using a set of recursive functions, each function handling a specific grammar rule, constructing a parse tree in the process. [3]

III. Design and Implementation

The index structure contains a map and one size type. The map stores every term in a document as its keys and another map as its values. The second map stores the document path as its keys and a query result type holding the the document path and the frequency of the term in the document as its values. The size type keeps track of the number of documents that has been added to the map.

When the program is run the `process_args()` reads the arguments and adds the paths to the documents to a list. The index is built using the `build_index()` function with the list with the documents as the argument. `build_index()` runs `index_create()` which allocates space for the index and initializes its structures. After creating the index, `build_index()` loops through the list of paths, tokenizes each document path with `read_file_terms()` and runs `index_document()` for each path.

`index_document()` takes the index, a document path and a list of all terms within that document as arguments. It starts by iterating through the list of terms and checking if a map has been created for the term. If not, a map is created for the term, with a duplicate of the document path as the key and a query result type holding a duplicate of the the document path and the frequency of the term in the document as the value. The newly created map is then inserted into the main map. If a map has already been created for the term, we check if the document path has been added to the terms map. If so, its frequency is increased by one and if not a duplicate of the document path is added to the map with one as the frequency. Finally the number of documents is increased by one.

After the index is built, the interpreter is started using `run_interpreter()`. `run_interpreter()` tokenizes the input using `tokenize_query()` and runs `execute_query()`. `execute_query()` runs `index_query()`. `index_query()` starts by building an abstract syntax tree with the `parse()` function, using an iterator for the list of tokens as its argument.

`parse()` starts a set of four recursive functions that build an abstract syntax tree from the list of tokens. The four functions are `parse_andnotterm()`, `parse_andterm()`, `parse_orterm()` and `parse_term()`. The functions are run in this order so that the abstract syntax tree has the right prioritization. The first three functions are very similar. They start by running the next function to determine the operators left child. Then they check if the token is a given operator. If so they determine the operators right child by running the next function on the next token. After this a node containing the right operator type and both the operators children is created and returned. The last function checks if the token is a parentheses and returns an abstract syntax tree of whatever the parentheses contains if this is the case. If the token is not a parentheses, the token is a word and a node containing the word is created and returned.

After `parse()` has created an abstract syntax tree, `evaluate_ast()` returns a set of all documents that match

the search. `evaluate_ast()` starts with the root node and checks if it is a word. If so, each document that contain the word and the frequency of the word in the document is put in a set and the set is returned. If not the function runs itself twice. The returned sets is the operators children. After this the appropriate operation is run on the sets depending on which operator the parent is. The remaining set is returned, containing every document path that matches the search and its frequency.

Every document path and its frequency is added to a list. The list is sorted and returned. After `index_query()` has returned the list, the results are processed using `process_query_results()`.

IV. Experiments and Results

```
real    0m0.949s
user    0m0.056s
sys     0m0.055s
```

Fig1: 100 files

```
real    0m4.746s
user    0m0.640s
sys     0m0.429s
```

Fig2: 1000 files

```
real    1m49.074s
user    0m6.092s
sys     0m5.289s
```

Fig3: 10000 files

V. Discussion

The results from the experiments are a bit unprecise as the timing had to be ended manually. Especially for 100 files the extra time this takes is significant in comparison to the time the program takes to run. We can see from the user and sys times that they are multiplied by ten when the number of files are multiplied by ten, but the actual time the indexing takes, although it is unprecise, seems to increase exponentially. Since a new map needs to be created for each term, indexing should have a time complexity of O(log n) as for each new document more terms have maps that has already been created.

The index's structure was chosen for simplicity and optimal lookup time. The implementation is quite straight forward compared to the alternatives and more importantly, lookup is close to instant as hashmaps have an average complexity of O(1) for lookup. Previously the structure was a map of lists containing query result types with the document path and the frequency. This structure had a significantly worse lookup time as the list needed to be traversed often.

To perform operations the documents that contain the term are inserted into sets in `evaluate_ast()`. To avoid this the documents could have been stored in sets instead of maps in `index_document()`, but this would lead to slower lookup time.

As of now the parser checks if the token is a parentheses, but it never checks if the parentheses has ended. Because of this everything after the first parentheses will be treated like it is inside the parentheses even if it is not. This should be fixed by checking for ")" in the functions and returning the parentheses ast if ")" is found.

VI. Conclusion

In this project we have implemented a search application. The search application does not support parentheses properly, but everything else should work as intended.

References

[1] GeeksforGeeks, "Inverted Index." [Online]. Available: https://www.geeksforgeeks.org/inverted-index/

[2] Wikipedia contributors, "Inverted Index." Wikipedia, The Free Encyclopedia, Mar. 2025.

[3] GeeksforGeeks, "Recursive Descent Parser," 2025, [Online]. Available: https://www.geeksforgeeks.org/recursive-descent-parser/